

**Федеральное государственное автономное образовательное
учреждение высшего образования «Национальный
исследовательский университет ИТМО»**

Факультет Программной Инженерии и Компьютерной Техники

Лабораторная работа №4
По Основам Программной Инженерии
Вариант 55147

Выполнил:

Ларионов Владислав Васильевич

Группа Р3209

Проверил:

Ермаков Михаил Константинович

Санкт-Петербург, 2026

Содержание

Задание	3
Выполнение задания №2.....	4
Выполнение задания №3.....	6
Выполнение задания №4.....	9
Программа	14
Вывод:	14

Задание

1. Для своей программы из [лабораторной работы #3](#) по дисциплине "Веб-программирование" реализовать:

- MBean, считающий общее число установленных пользователем точек, а также число точек, не попадающих в область. В случае, если координаты установленной пользователем точки вышли за пределы отображаемой области координатной плоскости, разработанный MBean должен отправлять оповещение об этом событии.
- MBean, определяющий площадь получившейся фигуры.

2. С помощью утилиты JConsole провести мониторинг программы:

- Снять показания MBean-классов, разработанных в ходе выполнения задания 1.
- Определить имя и версию ОС, под управлением которой работает JVM.

3. С помощью утилиты VisualVM провести мониторинг и профилирование программы:

- Снять график изменения показаний MBean-классов, разработанных в ходе выполнения задания 1, с течением времени.
- Определить имя класса, объекты которого занимают наибольший объём памяти JVM; определить пользовательский класс, в экземплярах которого находятся эти объекты.

4. С помощью утилиты VisualVM и профилировщика IDE NetBeans, Eclipse или Idea локализовать и устранить проблемы с производительностью в [программе](#). По результатам локализации и устранения проблемы необходимо составить отчёт, в котором должна содержаться следующая информация:

- Описание выявленной проблемы.
- Описание путей устранения выявленной проблемы.
- Подробное (со скриншотами) описание алгоритма действий, который позволил выявить и локализовать проблему.

Студент должен обеспечить возможность воспроизведения процесса поиска и локализации проблемы по требованию преподавателя.

Выполнение задания №2

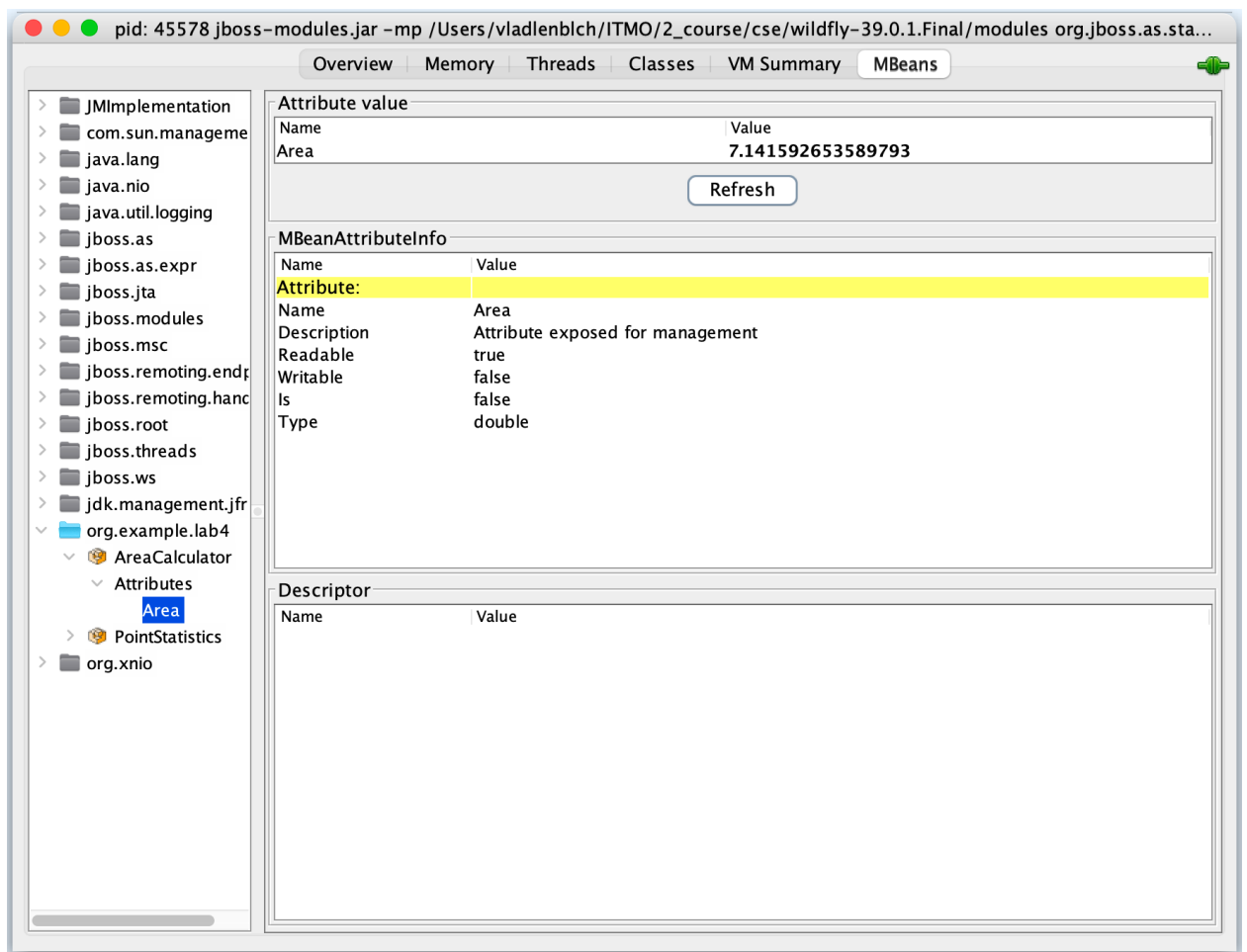


Рис. 2.1 – пример снятых показаний AreaCalculator

Далее скриншоты конкретно снятых показаний (без интерфейса JConsole)

Attribute value	
Name	Value
TotalPoints	9
Refresh	

Рис. 2.2 – пример снятых показаний PointStatistics для TotalPoints

Attribute value	
Name	Value
MissPoints	3
Refresh	

Рис. 2.3 – пример снятых показаний PointStatistics для MissPoints

Attribute value	
Name	Value
Name	Mac OS X

Рис. 2.3 – снятые показания имени ОС, под управлением которой работает JVM

Attribute value	
Name	Value
Version	26.3.1

Рис. 2.4 – снятые показания версии ОС, под управлением которой работает JVM

The screenshot shows the JBoss IDE interface with the 'Notification buffer' tab selected. The buffer contains three entries, with the most recent one highlighted in blue:

TimeStamp	Type	UserData	SeqNum	Message	Event	Source
20:13:03...	org.examp...		3	Point is out...	javax.man...	org.example.la...
20:12:56...	org.examp...		2	Point is out...	javax.man...	org.example.la...
20:12:24...	org.examp...		1	Point is out...	javax.man...	org.example.la...

The left sidebar shows the project structure with 'org.example.lab4' expanded, and 'Notifications[3]' selected under 'PointStatistics'. The bottom of the window has 'Subscribe', 'Unsubscribe', and 'Clear' buttons.

Рис. 2.5 – снятые оповещения о том, что точка не видна на графике

Выполнение задания №3

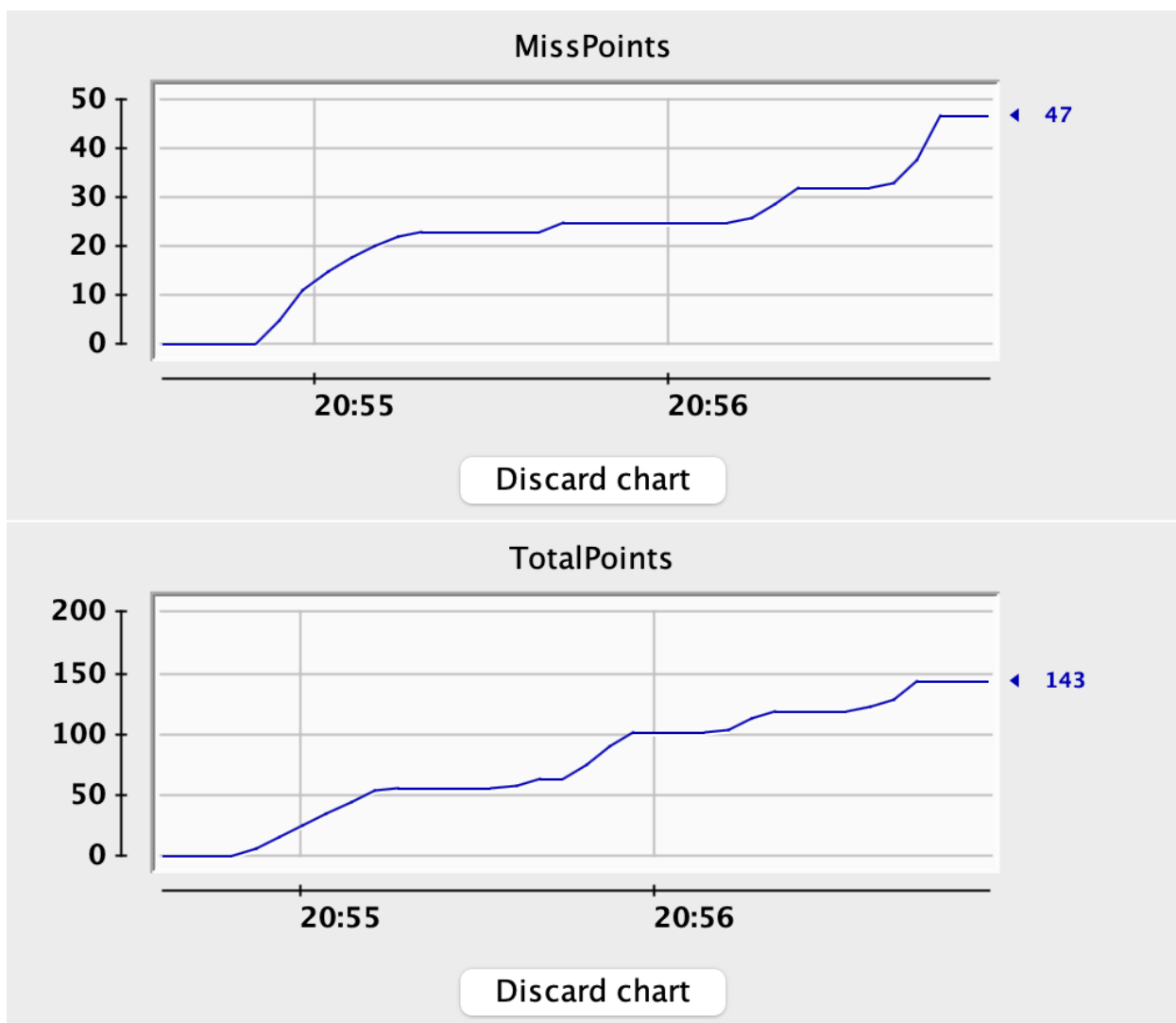


Рис. 3.1 – графики снятых показателей MBean-классов с течением времени

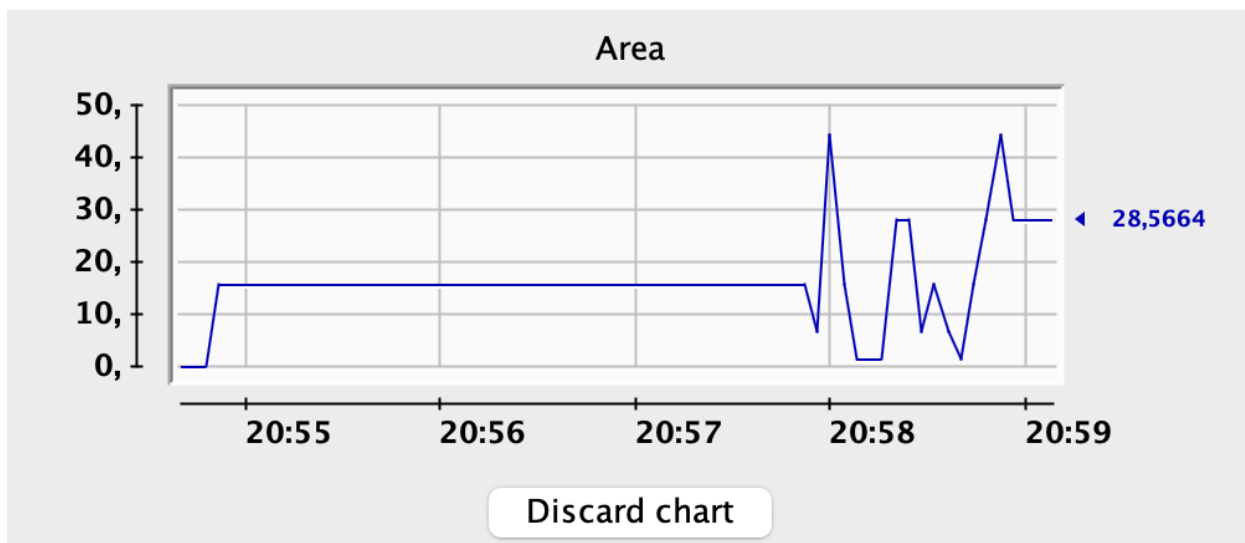


Рис. 3.2 – график снятых показателей MBean-классов с течением времени

Profiler Snapshot			
Name	Live Bytes		Live Objects
byte[]	32 851 672 B (24,8 %)		340 142 (11,1 %)
java.util.HashMap\$Node	9 664 864 B (7,3 %)		302 027 (9,8 %)
java.lang.Object[]	7 901 600 B (6 %)		243 980 (7,9 %)
java.lang.String	6 608 616 B (5 %)		275 359 (9 %)
java.util.HashMap\$Node[]	6 278 696 B (4,7 %)		75 259 (2,4 %)
int[]	4 816 160 B (3,6 %)		40 241 (1,3 %)
java.util.HashMap	4 615 392 B (3,5 %)		96 154 (3,1 %)
java.util.HashMap\$KeyIterator	4 104 800 B (3,1 %)		102 620 (3,3 %)
char[]	2 863 960 B (2,2 %)		58 672 (1,9 %)
java.lang.Class	2 527 376 B (1,9 %)		21 072 (0,7 %)
java.lang.reflect.Type[]	1 601 752 B (1,2 %)		74 583 (2,4 %)
java.lang.reflect.Method	1 587 168 B (1,2 %)		18 036 (0,6 %)
java.nio.HeapCharBuffer	1 578 920 B (1,2 %)		28 195 (0,9 %)
java.util.ArrayList	1 532 088 B (1,2 %)		63 837 (2,1 %)
java.util.HashMap\$EntryIterator	1 369 640 B (1 %)		34 241 (1,1 %)
java.util.TreeMap\$Entry	1 326 040 B (1 %)		33 151 (1,1 %)
java.lang.reflect.Parameter	1 203 240 B (0,9 %)		30 081 (1 %)

Рис. 3.3 – снимок результата распределения памяти

Heap Dump			
Name	Count	Size	Retained (sort to get)
byte[]	222 394 (14,5 %)	25 026 776 B (34,7 %)	n/a
byte[]#56935 : 789 010 items		789 032 B (1,1 %)	n/a
<items>			
<references>			
cen in java.util.zip.ZipFile\$Source#414		80 B (0 %)	n/a
byte[]#48408 : 782 745 items		782 768 B (1,1 %)	n/a
<items>			
<references>			
cen in java.util.zip.ZipFile\$Source#328		80 B (0 %)	n/a
value in java.util.HashMap\$Node#49186		32 B (0 %)	n/a
zsrc in java.util.zip.ZipFile\$CleanableResource#719		32 B (0 %)	n/a
zsrc in java.util.zip.ZipFile\$CleanableResource#720		32 B (0 %)	n/a
byte[]#163461 : 680 384 items		680 400 B (0,9 %)	n/a

Рис. 3.4 – распределение памяти по классу byte[]

Heap Dump			
Name	Count	Size	Retained (sort to get)
org.example.mbeans.PointStatistics	2 (0 %)	80 B (0 %)	n/a
org.example.service.DatabaseService\$Proxy\$_\$\$_WeldClientProxy	1 (0 %)	48 B (0 %)	n/a
org.example.mbeans.MBeanRegistry\$Proxy\$_\$\$_WeldClientProxy	1 (0 %)	48 B (0 %)	n/a
org.example.mbeans.AreaCalculator	2 (0 %)	48 B (0 %)	n/a
org.example.beans.UserBean\$Proxy\$_\$\$_WeldClientProxy	1 (0 %)	40 B (0 %)	n/a
org.example.beans.PointBean	1 (0 %)	40 B (0 %)	n/a
org.example.beans.ResultsBean\$Proxy\$_\$\$_WeldClientProxy	1 (0 %)	32 B (0 %)	n/a
org.example.mbeans.MBeanRegistry	1 (0 %)	32 B (0 %)	n/a
org.example.service.DatabaseService	1 (0 %)	32 B (0 %)	n/a
org.example.entities.UserEntity	1 (0 %)	24 B (0 %)	n/a
org.example.beans.UserBean	1 (0 %)	24 B (0 %)	n/a
org.example.beans.ClockBean	1 (0 %)	24 B (0 %)	n/a
org.example.beans.ResultsBean	1 (0 %)	16 B (0 %)	n/a
org.example.mbeans.interfaces.AreaCalculatorMBean	0 (0 %)	0 B (0 %)	n/a
org.example.mbeans.interfaces.PointStatisticsMBean	0 (0 %)	0 B (0 %)	n/a
org.example.entities.PointEntity	0 (0 %)	0 B (0 %)	n/a
org.example.validation.RValidator	0 (0 %)	0 B (0 %)	n/a
org.example.validation.YValidator	0 (0 %)	0 B (0 %)	n/a
org.example.validation.XValidator	0 (0 %)	0 B (0 %)	n/a

Рис. 3.5 – распределение памяти среди пользовательских классов

По данным Memory Sampler VisualVM наибольший объем памяти среди всех классов занимают объекты класса byte[]: 29 761 008 B, 302 135 объектов.

Для определения владельцев этих объектов был построен Heap Dump. Анализ показал, что крупнейшие byte[] удерживаются объектами java.util.zip.ZipFile\$Source. Следовательно, эти объекты относятся к инфраструктуре JVM/WildFly и загруженным JAR/ZIP-ресурсам, а не к пользовательским классам приложения.

Пользовательского класса `org.example.*`, в экземплярах которого находились бы крупнейшие `byte[]`, в Heap Dump обнаружено не было. Среди пользовательских классов приложения наибольший собственный размер имеет `org.example.mbeans.PointStatistics`: 2 объекта, 80 В.

Выполнение задания №4

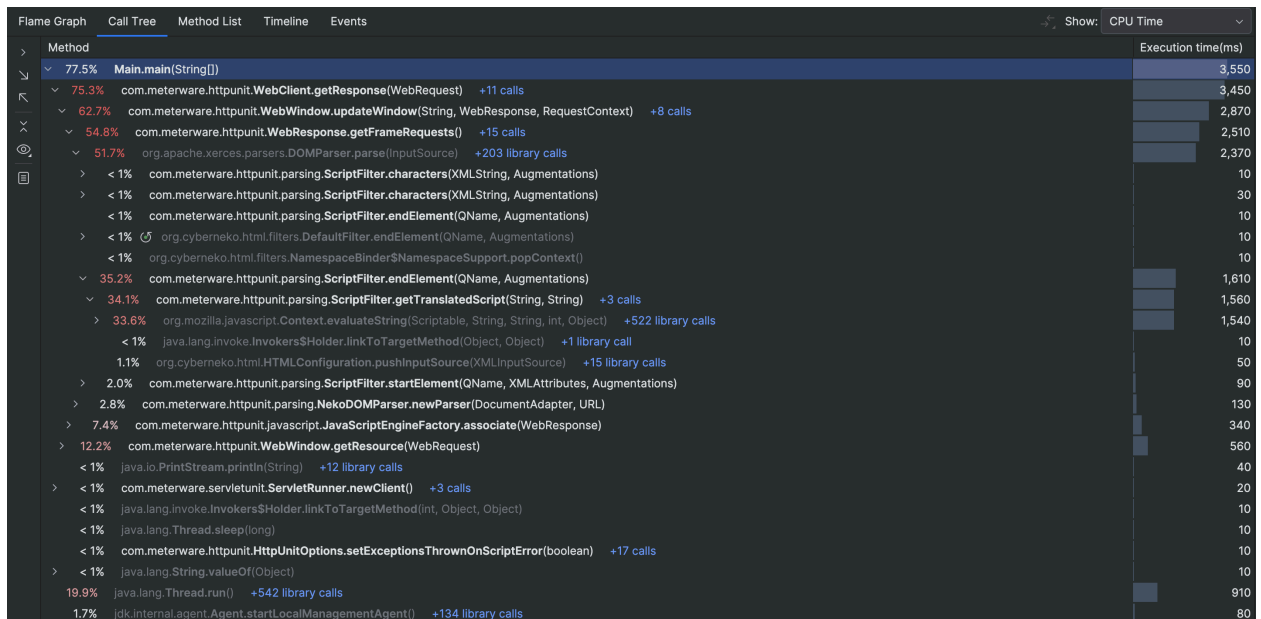


Рис. 4.1 – Call Tree по CPU Time в IntelliJ Idea

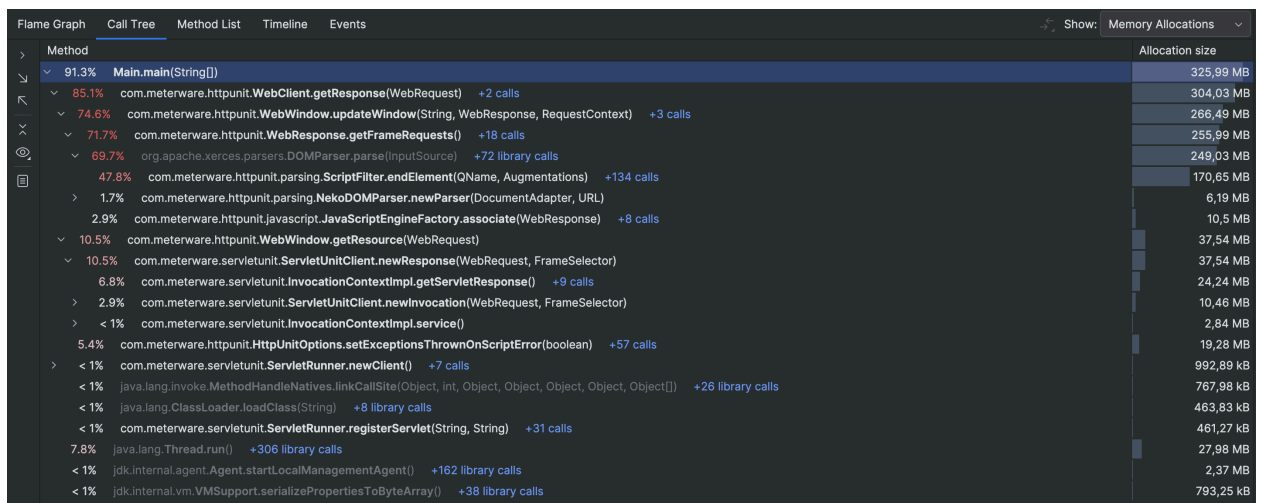


Рис. 4.2 – Call Tree по Memory Allocations в IntelliJ Idea

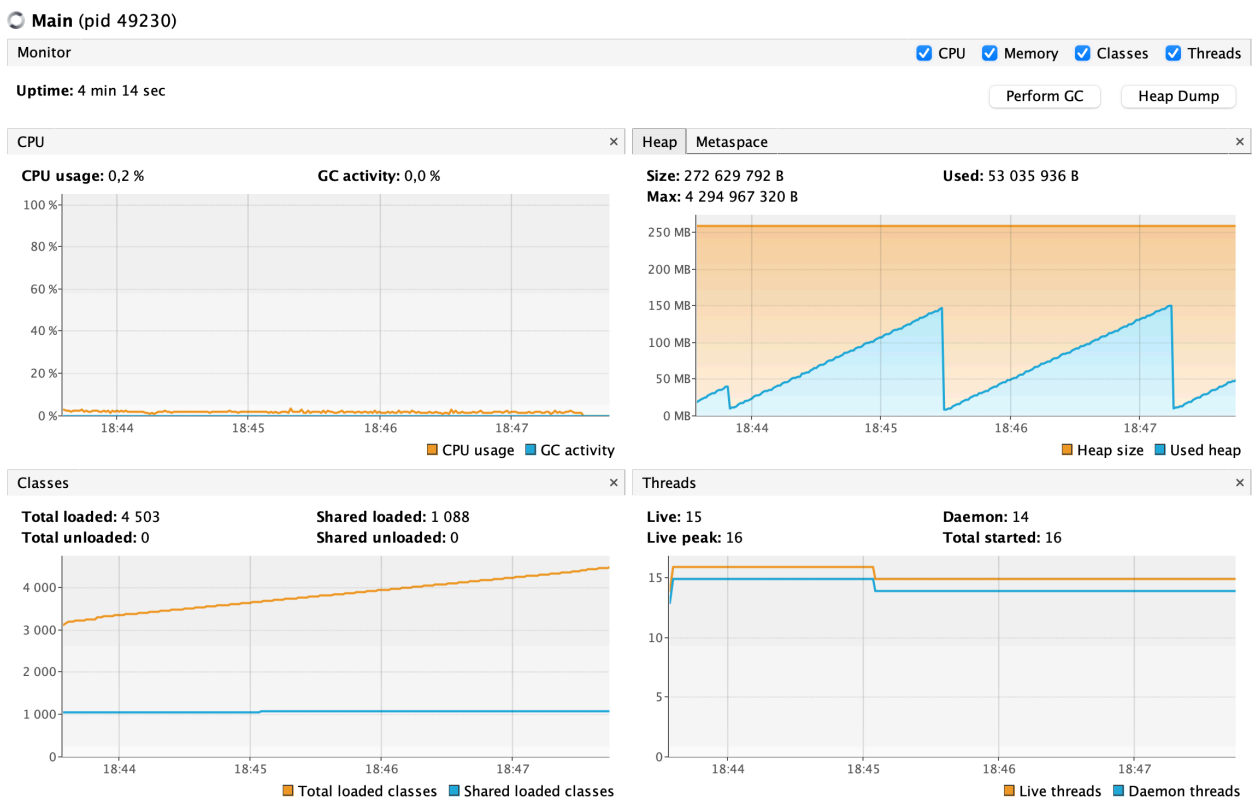


Рис. 4.3 – показатели до исправления

Основной подозрительный признак удалось найти на графике Classes: количество загруженных классов постоянно увеличивается. Это указывает на возможную проблему с Metaspace или динамической загрузкой классов.

Основная проблема – вызов `ServletUnitClient.getResponse(...)`, внутри которого `HttpUnit` разбирал HTML-ответ сервлета. Далее профилировщик показал переход к `ScriptFilter.getTranslatedScript(...)` и `Context.evaluateString(...)`, то есть к обработке JavaScript.

После проверки `HelloWorld.java` было найдено, что сервлет возвращает JavaScript:

```
document.wr('Hello Document')
```

В коде была ошибка: вместо `document.write(...)` использовался несуществующий метод `document.wr(...)`. Кроме того, сам JavaScript был лишним, так как он просто выводил статический текст. Из-за этого `HttpUnit` при каждом запросе пытался обработать скрипт, что приводило к лишним аллокациям и росту количества загруженных классов.

Исправление:

Проблема была устранена на двух уровнях: в ответе сервлета и в настройках клиента.

В HelloWorld.java JavaScript был заменён на обычный HTML:

```
out.println("<P>Hello Document</P>");
```

Так как текст статический, использование JavaScript здесь не требуется. После этой замены HttpUnit.

Дополнительно в Main.java было отключено выполнение JavaScript на стороне клиента:

```
HttpUnitOptions.setExceptionsThrownOnScriptError(false);  
HttpUnitOptions.setScriptingEnabled(false);
```

Такое решение было принято потому, что программа не проверяет работу JavaScript, а только многократно получает HTML-ответ сервлета. Поэтому выполнение скриптов в HttpUnit является лишней операцией. Замена скрипта устраняет источник проблемы, а отключение JavaScript на клиенте защищает программу от повторения такой ситуации.

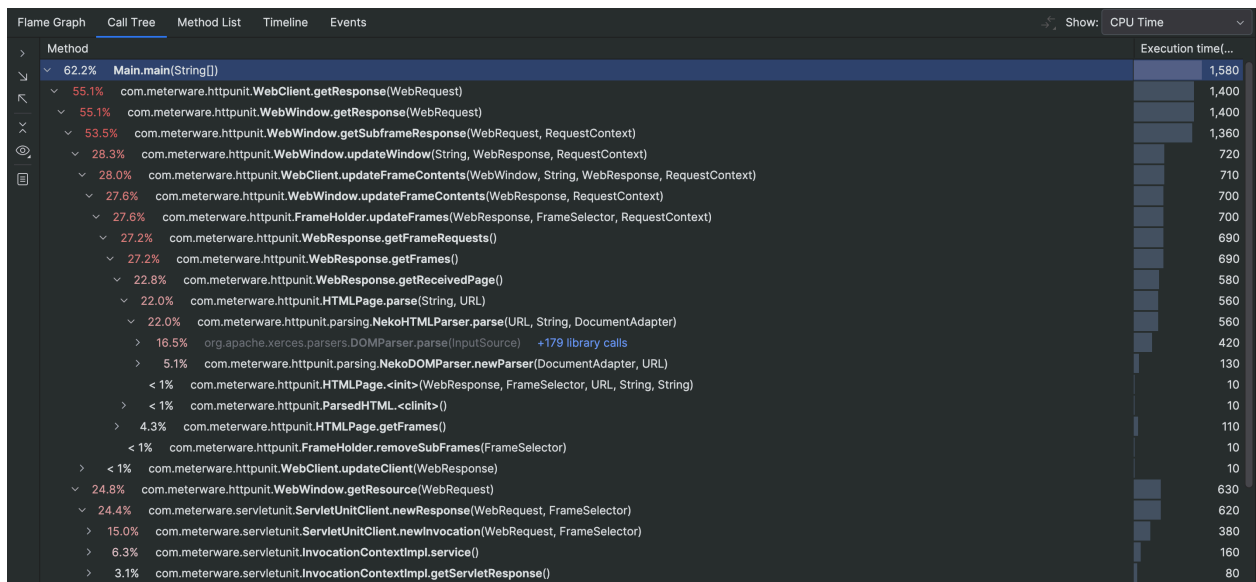


Рис. 4.4 – Call Tree по CPU Time в IntelliJ Idea

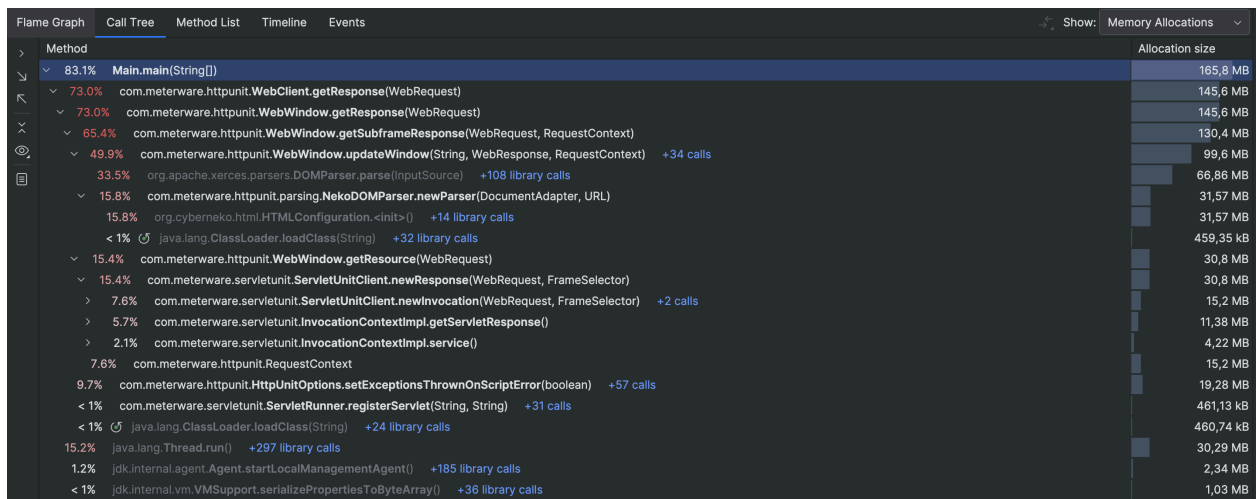


Рис. 4.5 – Call Tree по Memory Allocations в IntelliJ Idea

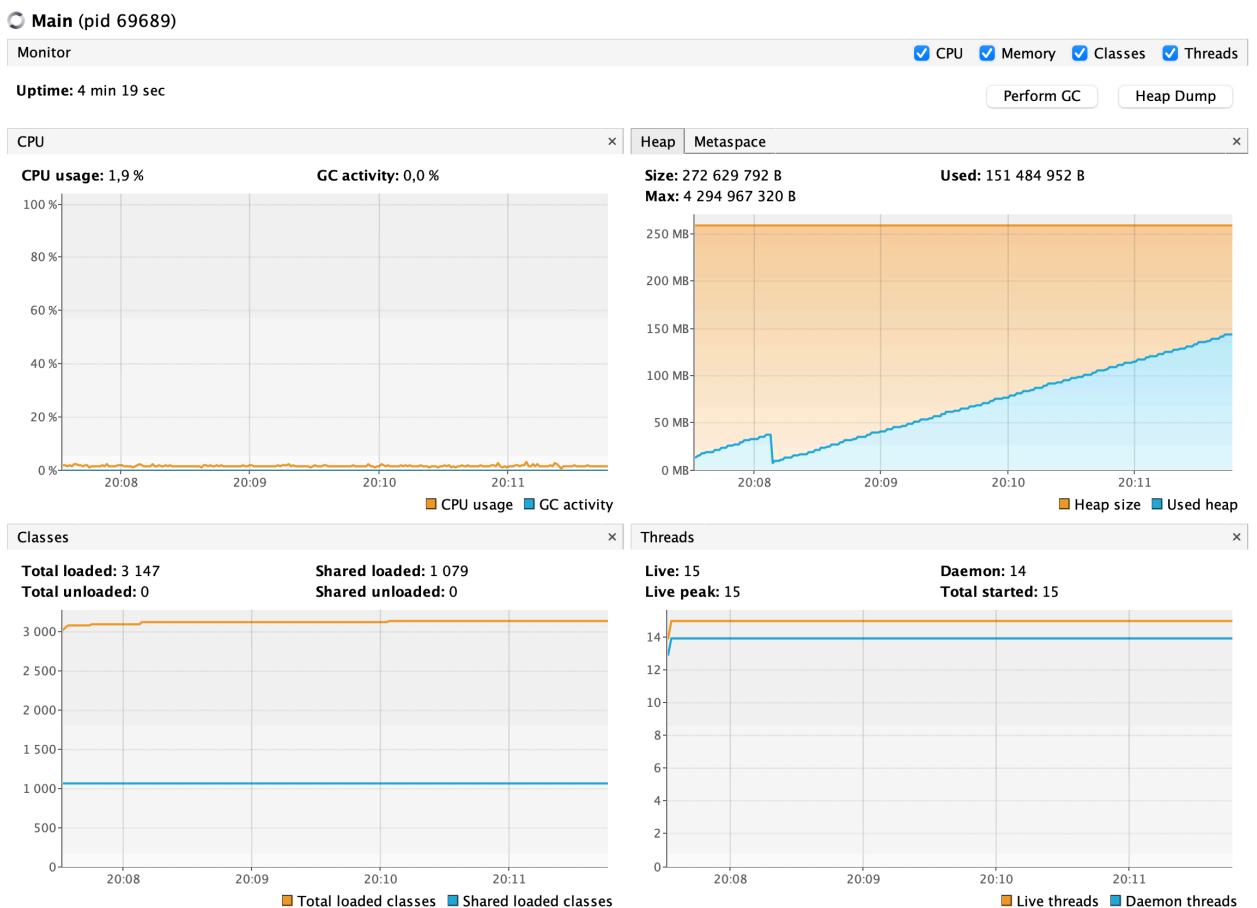


Рис. 4.6 – показатели после исправления

После внесения изменений программа была повторно проверена в VisualVM. На графике Classes видно, что количество загруженных классов после небольшого роста при старте стабилизировалось примерно на уровне 3147 и дальше почти не увеличивалось. Это главное отличие от состояния до исправления, где число классов постоянно росло. Следовательно, проблема с динамической загрузкой классов была устранена.

График Heap показывает постепенный рост используемой памяти, но это связано с созданием временных объектов при обработке запросов и не указывает на прежнюю проблему с загрузкой классов.

Повторное профилирование показало, что проблемные методы, связанные с обработкой JavaScript, больше не являются значимыми:

```
ScriptFilter.getTranslatedScript(...)
Context.evaluateString(...)
```

После исправления основная работа программы сводится к стандартной обработке HTTP-ответа.

Также уменьшились аллокации памяти: Main.main() стал выделять около 165.8 MB вместо 325.99 MB, а ServletUnitClient.getResponse(...) — около 145.6 MB вместо 303.03 MB.

Таким образом, причина была определена верно: проблему вызывала обработка JavaScript в HTML-ответе сервлета. После удаления скрипта и отключения JavaScript в HttpUnit рост количества классов прекратился, а потребление ресурсов стало стабильнее.

Программа

Исходный код проекта лежит в публичном GitHub-репозитории по ссылке:

https://github.com/vladlenblch/ITMO_VT/tree/main/2_course/cse/lab4

Вывод:

Во время выполнения данной лабораторной работы я:

- 1) Повторил навыки профилирования
- 2) Повторил навыки поиска багов и их исправления
- 3) Научился работать с JConsole
- 4) Научился работать с VisualVM
- 5) Научился работать с профилировщиком IntelliJ Idea